

Machine Problem: MIPS64 Memory Assignment and Load-/Store Operations

Given

The 16 consecutive memory locations below (addresses 0000 to 000F, shown in hexadecimal) constitute a single memory block `MEM[0..15]` in EduMIPS64. Each location is **8 bits (1 byte)** wide, since the EduMIPS64 simulator uses a byte-addressable memory model.

Define labels:

$$\text{ALPHA} \equiv \text{MEM} + 0, \quad \text{BETA} \equiv \text{MEM} + 8.$$

Addr (offset or index)	Byte	Addr (offset or index)	Byte
0000	80	0008	01
0001	91	0009	12
0002	A2	000A	23
0003	B3	000B	34
0004	C4	000C	45
0005	D5	000D	56
0006	E6	000E	67
0007	F7	000F	78

Objectives

1. **Dynamically assign** (at run time) the 16 consecutive **8-bit values (bytes)** shown above into MIPS memory.
2. Perform the following loads (use `$s1..$s4` as `r1..r4`):
 - (a) **LB** into `r1 ($s1)` from `ALPHA+4`.
 - (b) **LBU** into `r2 ($s2)` from `ALPHA+7`.
 - (c) **LW** into `r3 ($s3)` from `ALPHA+0` (aligned word).
 - (d) **LD** into `r4 ($s4)` from `ALPHA+0` (aligned doubleword).
3. **Stores**
 - (e) Copy the sequence of bytes from `ALPHA[index]` where $3 < \text{index} < 5$ into `OMEGA`, beginning at `OMEGA[0]`.
 - (f) From the memory location in `BETA` corresponding to one half of the index used in item (e) as the starting point, copy a sequence of 16 bits (two consecutive 8-bit values) into `LAMBDA`, beginning at `LAMBDA+0`.

What to submit

- Write your entire MIPS64 source code on one whole sheet of yellow paper if you believe it is correct after performing a simulation on your machine.